💾 CATEGORY 6: Storage & I/O — Complete Deep Dive

6.1 `df`, `du` — Disk Usage, Inode Exhaustion

🔷 What they are in simple terms

`df` (**d**isk **f**ree) tells you how much space is left on each **filesystem**. `du` (**d**isk **u**sage) tells you how much space a specific **directory or file** consumes. They answer two different questions: `df` = "how full is my disk?", `du` = "what's eating my disk?"

🔷 How Linux tracks disk space — Two systems

```
Every filesystem has TWO things that can be exhausted:

1. DATA BLOCKS — the actual space to store file contents
   └── tracked by: df -h (shows Used/Available)

2. INODES — metadata slots (one per file/directory)
   └── tracked by: df -i
   └── Each inode stores: permissions, owner, timestamps, block pointers
   └── NOT the filename (filenames live in directory entries)

┌──────────────────────────────────────────────────────────┐
│  You CAN run out of inodes even with space available │
│  You CAN run out of space even with inodes available │
│  Both must be monitored in production!               │
└──────────────────────────────────────────────────────────┘
```

🔷 `df` — Filesystem-level disk usage

```
# ── Basic usage ──────────────────────────────────────────────

# Human readable (KB/MB/GB)
df -h
# Filesystem      Size  Used Avail Use%  Mounted on
# /dev/sda1        50G   35G   13G  73%  /
# /dev/sdb1       500G  450G   50G  90%  /data
# tmpfs           7.8G  1.2G  6.6G  16%  /dev/shm
# overlay          50G   35G   13G  73%  /var/lib/docker/overlay2/abc123

# -H: Human readable with 1000 (not 1024) as base
df -H
```

```
# Specific filesystem or path
df -h /data          # Filesystem containing /data
df -h /dev/sda1      # Specific device

# Show filesystem type
df -hT
# Filesystem     Type   Size  Used Avail Use% Mounted on
# /dev/sda1      ext4    50G   35G   13G  73% /
# /dev/sdb1      xfs    500G  450G   50G  90% /data
# tmpfs          tmpfs  7.8G  1.2G  6.6G  16% /dev/shm

# ── Inode usage — the hidden problem ──────────────────────────────

df -i
# Filesystem      Inodes  IUsed   IFree IUse% Mounted on
# /dev/sda1      3276800  312450 2964350   10% /
# /dev/sda2      1638400 1638399       1  100% /var/spool  ← FULL! No
space to create files!

df -ih           # Human readable inode counts

# ── Production patterns ───────────────────────────────────────────

# Alert when disk > 80% full
df -h | awk 'NR>1 && $5+0 > 80 {print "WARNING:", $0}'

# Alert on inode exhaustion
df -i | awk 'NR>1 && $5+0 > 80 {print "INODE WARNING:", $0}'

# Quick one-liner: show only filesystems over threshold
df -h | awk 'NR==1 || $5+0 >= 80'

# Exclude tmpfs and overlay (focus on real disks)
df -h --exclude-type=tmpfs --exclude-type=overlay

# Watch disk usage over time
watch -n 5 'df -h'

# ── What Use% 100 means ───────────────────────────────────────────
# Regular files: CANNOT write new data
# root user: has ~5% reserved space (tune with tune2fs -m)
# So Use% can show 100% for users but root can still write
# tune2fs -l /dev/sda1 | grep "Reserved block count"
```

🔷 du — Directory-level disk usage

```
# ── Basic usage ───────────────────────────────────────────────────

# Size of current directory (recursive)
du -sh .
```

```
# 2.3G    .

# Size of specific directory
du -sh /var/log
# 456M    /var/log

# All subdirectories, human readable, summarized
du -sh /var/*/
# 12M     /var/backups/
# 456M    /var/log/
# 2.1G    /var/lib/
# 8.0K    /var/mail/

# ── Finding what's large ──────────────────────────────────────

# Top 10 largest directories under /var
du -h /var/* 2>/dev/null | sort -rh | head -10

# Top 10 largest files anywhere on disk
find / -type f -printf '%s %p\n' 2>/dev/null | sort -rn | head -10

# Larger than 100MB files
find /var -type f -size +100M -exec ls -lh {} \;

# ── Inode count (number of files) ────────────────────────────

# Count files per directory (to find inode hogs)
du --inodes -s /var/*/
# 12      /var/backups/
# 8420    /var/log/
# 98432   /var/lib/    ← Many small files!

# ── Excluding paths ───────────────────────────────────────────

# Exclude certain filesystems (don't cross mount points)
du -sh --one-file-system /var

# Exclude specific directory
du -sh --exclude=/var/lib/docker /var

# ── Production incident: disk full ────────────────────────────

# Step 1: Which filesystem is full?
df -h | awk '$5+0 >= 90'

# Step 2: What's using the space?
du -sh /var/*/  2>/dev/null | sort -rh | head -20

# Step 3: Drill down
du -sh /var/log/* | sort -rh | head -10

# Step 4: Find large individual files
find /var/log -type f -size +50M -ls
```

```
# Step 5: Check for deleted-but-open files still consuming space
lsof +L1 | grep -i del
# (files deleted but still held open by a process — space not freed!)
# Fix: restart the process that has the file open

# Step 6: Clean up
# Truncate log (safe — preserves inode, clears content)
> /var/log/app.log
# Or: : > /var/log/app.log
# Or: truncate -s 0 /var/log/app.log

# DO NOT: rm /var/log/app.log (if process has it open, space not freed!)
# DO: truncate the file while process keeps fd open

# ── Inode exhaustion: diagnosing ──────────────────────────────────────

# Find directories with MANY files (inode hogs)
for dir in /var/spool /tmp /var/lib; do
    count=$(find "$dir" -maxdepth 2 | wc -l)
    echo "$count $dir"
done | sort -rn | head

# Common inode exhaustion causes:
# - /var/spool/postfix/  ← Mail queue overflow (undelivered mail)
# - /tmp/                ← Temp files not cleaned up
# - PHP session dirs     ← /var/lib/php/sessions/
# - Node.js projects     ← node_modules/ has MILLIONS of files
# - Log rotation failure ← thousands of rotated log files
```

🔷 Short crisp interview answer

> "`df -h` shows filesystem-level free space; `du -sh /path` shows what a directory consumes. There are two things that can be exhausted: data blocks and inodes. `df -i` shows inode usage — you can have space available but no inodes left, which prevents creating new files even though `df -h` looks fine. To find what's eating disk: `du -sh /var/*/ | sort -rh | head`. For deleted-but-open files still consuming space, `lsof +L1` finds processes holding deleted file descriptors — restarting those processes frees the space."

⚠ Gotchas

```
# GOTCHA 1: File deleted but space not freed
rm /var/log/app.log    # File deleted — but if nginx still has it open...
df -h                  # Shows same usage! Space not freed.
lsof +L1               # Lists deleted files still open
# Fix: kill -USR1 nginx (triggers log reopen) OR restart the process
# Fix without restart: > /proc/PID/fd/N  — truncates via live fd directly;
# inode stays valid from process's perspective, no crash needed
```

```
# GOTCHA 2: Inode exhaustion with space available
df -h    # Shows 40% used — looks fine
touch /tmp/newfile    # touch: cannot touch '/tmp/newfile': No space left
on device
df -i    # Shows 100% inode usage — THAT's the real problem

# GOTCHA 3: du vs df discrepancy
du -sh /    # Shows 20GB
df -h /     # Shows 35GB used
# Reason: du counts file content; df counts allocated blocks
# Sparse files, deleted-open files, and filesystem overhead cause
differences

# GOTCHA 4: du on NFS/bind mounts
du -sh /    # Can double-count if / has bind mounts
du -sh --one-file-system /    # Stays on one filesystem only

# GOTCHA 5: Reserved blocks (root-only space)
df -h       # Shows 100% for users
# Root user can still write — 5% blocks reserved by default for ext4
# Tune: tune2fs -m 1 /dev/sda1   (reduce to 1%)

# GOTCHA 6: du vs df discrepancy has exactly two root causes — always
check both:
# lsof +L1 for deleted-but-open files
# df -i for inode exhaustion
# These are the two diagnostic commands to run before anything else when
du and df disagree.
```

6.2 `mount` / `umount` — Mounting Filesystems, `/etc/fstab`

🔷 What mounting means

Linux uses a **single unified directory tree** starting at `/`. Every disk, partition, network share, or virtual filesystem is **attached** (mounted) at a **mount point** — a directory in that tree. Before mounting, the device's contents are invisible. After mounting, they appear as regular directories.

🔷 How mount works internally

```
Physical world:          Linux filesystem tree:
                                /
/dev/sda1 (ext4) ──────────────→/          (root filesystem)
/dev/sdb1 (xfs)  ──────────────→/data      (data disk)
//server/share   ──────────────→/mnt/nas   (NFS/CIFS share)
tmpfs            ──────────────→/tmp       (RAM-backed filesystem)
```

```
proc              ──────────────►/proc      (kernel info, virtual)
sysfs             ──────────────►/sys       (device/driver info)

mount() syscall:
1. Kernel finds block device driver for filesystem type
2. Driver reads superblock (filesystem metadata)
3. Kernel attaches filesystem root to the mount point directory
4. VFS (Virtual Filesystem Switch) routes all I/O through the driver
```

🔷 `mount` — Core usage

```
# ── View current mounts ──────────────────────────────────────────

# Simple listing
mount
# /dev/sda1 on / type ext4 (rw,relatime,errors=remount-ro)
# /dev/sdb1 on /data type xfs (rw,relatime,attr2,inode64)
# tmpfs on /tmp type tmpfs (rw,nosuid,nodev)
# proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)

# Cleaner output
findmnt
findmnt --real    # Only real filesystems (no proc/sys/cgroup noise)
findmnt -t ext4,xfs   # Specific types

# ── Mount a device ───────────────────────────────────────────────

# Basic mount (kernel auto-detects filesystem type)
sudo mount /dev/sdb1 /mnt/data

# Specify filesystem type explicitly
sudo mount -t ext4 /dev/sdb1 /mnt/data
sudo mount -t xfs  /dev/sdb1 /mnt/data
sudo mount -t nfs  server:/export /mnt/nfs

# Mount with options
sudo mount -o ro /dev/sdb1 /mnt/data        # Read-only
sudo mount -o rw,noatime /dev/sdb1 /mnt/data # Read-write, no access time
updates
sudo mount -o remount,rw /                  # Remount root read-write

# ── Mount options explained ──────────────────────────────────────

# rw/ro          → read-write or read-only
# noatime        → don't update access time (HUGE performance improvement)
# nodiratime     → don't update directory access time
# noexec         → don't allow executing binaries from this mount
# nosuid         → ignore setuid/setgid bits (security)
# nodev          → don't interpret character/block devices
# relatime       → update atime only if newer than mtime (compromise)
```

```
# sync           → writes are synchronous (slower, safer)
# async          → writes are asynchronous (faster, default)
# user           → allow non-root users to mount this
# users          → allow any user to mount and unmount
# auto/noauto    → mount at boot / don't auto-mount
# defaults       → rw,suid,dev,exec,auto,nouser,async

# ── Loop mounts (mount disk image as filesystem) ──────────────────

sudo mount -o loop disk.img /mnt/image      # Mount ISO or disk image
sudo mount -t iso9660 -o loop file.iso /mnt/cdrom

# ── Bind mounts (mount directory at another path) ─────────────────

sudo mount --bind /var/www /srv/web         # Same content, different path
sudo mount -o bind /source /destination     # Same thing

# ── tmpfs (RAM-backed filesystem) ──────────────────────────────────

sudo mount -t tmpfs -o size=512m tmpfs /mnt/ramdisk
# Useful for: fast temp space, /tmp, build caches
# Disappears on reboot — don't store persistent data here

# ── NFS mount ──────────────────────────────────────────────────────

sudo mount -t nfs -o rw,hard,intr server.local:/export/data /mnt/nfs
# hard = retry indefinitely if server unreachable
# intr = allow interrupt (Ctrl+C) during hard retry
# soft = fail after timeout (can cause data loss)

# ── Check what's mounted where ─────────────────────────────────────

cat /proc/mounts        # Kernel's actual view of mounts (authoritative)
cat /proc/self/mounts   # Same but for current process's namespace
```

🔷 `umount` — Unmounting

```
# Basic unmount (by mount point or device)
sudo umount /mnt/data
sudo umount /dev/sdb1

# Lazy unmount — detach from filesystem tree now, clean up when idle
sudo umount -l /mnt/data
# Good for NFS that won't disconnect cleanly

# Force unmount (dangerous — can cause data loss)
sudo umount -f /mnt/nfs

# ── "Target is busy" errors ────────────────────────────────────────
sudo umount /mnt/data
```

```
# umount: /mnt/data: target is busy

# Find what's using the mount point
lsof +D /mnt/data          # Files opened under this path
fuser -mv /mnt/data        # Processes using the mount

# Kill processes using it (careful!)
fuser -km /mnt/data        # -k = kill, -m = mount point

# Or: cd out of it if your shell is there
cd /tmp && sudo umount /mnt/data
```

🔷 `/etc/fstab` — Persistent mount configuration

```
# Format:
# <device>         <mount point>  <type>   <options>          <dump>
<pass>
# /dev/sda1        /              ext4     defaults,errors=remount-ro  0
1
# /dev/sdb1        /data          xfs      defaults,noatime   0  2
# UUID=abc123      /backup        ext4     defaults,nofail    0  2
# server:/export   /mnt/nfs       nfs      rw,hard,intr,auto  0  0
# tmpfs            /tmp           tmpfs    defaults,size=1g   0  0
# LABEL=mydisk     /mnt/mydisk    ext4     defaults           0  2

# dump field: 0 = don't backup with dump command (use 0 always now)
# pass field: fsck order at boot
#   0 = don't fsck
#   1 = fsck first (root filesystem only)
#   2 = fsck after root (other filesystems)

# ── Using UUID instead of device names ──────────────────────────

# Device names (/dev/sda1) can CHANGE between boots!
# UUIDs are stable — always use them in fstab

# Find UUID of a device
blkid /dev/sdb1
# /dev/sdb1: UUID="550e8400-e29b-41d4-a716-446655440000" TYPE="xfs"

lsblk -f
# NAME   FSTYPE  LABEL  UUID                                 MOUNTPOINT
# sda
# └─sda1 ext4           550e8400-e29b-41d4-a716-446655440000 /

# fstab entry with UUID:
UUID=550e8400-e29b-41d4-a716-446655440000  /data  xfs  defaults,noatime  0
2

# ── Important fstab options ─────────────────────────────────────
```

```
# nofail: don't fail boot if device is absent (for external/removable)
UUID=abc123  /mnt/backup  ext4  defaults,nofail  0  2

# _netdev: wait for network before mounting (for NFS, CIFS)
server:/export  /mnt/nfs  nfs  rw,hard,_netdev  0  0

# ── Apply fstab changes ──────────────────────────────────────────

# Mount everything in fstab not yet mounted
sudo mount -a

# Verify fstab is valid before reboot
sudo mount -a --dry-run   # Not all distros support this
sudo findmnt --verify     # Verify fstab entries

# ── Danger zone ──────────────────────────────────────────────────
# Wrong fstab entry → system fails to boot!
# Always:
# 1. Test with: sudo mount -a  BEFORE rebooting
# 2. Use nofail for non-critical mounts
# 3. Keep a rescue USB handy when editing fstab on remote servers

# ── systemd mount units (modern alternative to fstab) ────────────

# /etc/systemd/system/mnt-data.mount
# [Unit]
# Description=Data disk
# After=local-fs.target
#
# [Mount]
# What=/dev/disk/by-uuid/550e8400-e29b-41d4-a716-446655440000
# Where=/mnt/data
# Type=xfs
# Options=defaults,noatime
#
# [Install]
# WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable --now mnt-data.mount
```

🔷 Short crisp interview answer

> "Mounting attaches a filesystem to a directory (mount point) in the unified Linux directory tree. The VFS layer routes all I/O to the appropriate filesystem driver. I always use UUIDs in `/etc/fstab` instead of device names because `/dev/sda1` can change between boots. Critical fstab options: `noatime` for performance (skips access time writes), `nofail` for non-critical devices so they don't break boot, and `_netdev` for network filesystems to wait for network initialization. After editing fstab, I test with `sudo mount -a` before rebooting."

⚠ Gotchas

```
# GOTCHA 1: Bad fstab breaks boot
# Test: sudo mount -a  before any reboot
# Recovery: boot to rescue mode, mount root manually, fix /etc/fstab

# GOTCHA 2: Umount fails with "device is busy"
# Most common cause: your shell's CWD is inside the mount
cd /tmp    # Move out first
sudo umount /mnt/data

# GOTCHA 3: Lazy umount (-l) hides problems
sudo umount -l /mnt/nfs   # Returns success immediately
# But processes still have files open — they'll get I/O errors
# Use lazy umount only as a last resort for stuck NFS mounts

# GOTCHA 4: Mount options don't persist without fstab
sudo mount -o noatime /dev/sdb1 /data   # Works now
# After reboot: mounted WITHOUT noatime (back to defaults)
# Fix: add the option to /etc/fstab

# GOTCHA 5: noatime vs relatime
# noatime: never update atime — maximum performance
# relatime: update atime only if mtime/ctime is newer (default on modern
kernels)
# relatime is the compromise that satisfies most apps
# Some apps (mutt, tmpwatch) rely on atime — test before using noatime
```

6.3 Filesystem Types — ext4, xfs, btrfs, tmpfs — Trade-offs

🔷 What a filesystem is

A filesystem is the **organization system** for data on a block device. It defines: how files and directories are structured, where metadata lives, how free space is tracked, and what guarantees are made about data integrity.

🔷 The filesystem landscape

```
┌──────────────────────────────────────────────────────────────┐
│ Filesystem    │ Best for              │ Key strength          │
├──────────────────────────────────────────────────────────────┤
│ ext4          │ General purpose       │ Stable, universal, battle- │
│               │ OS root, most servers │ tested, fast fsck     │
├──────────────────────────────────────────────────────────────┤
```

```
│ xfs          │ Large files, high I/O │ Parallel I/O, huge files,  │
│              │ databases, media      │ no fsck downtime           │
├──────────────┼──────────────────────┼────────────────────────────┤
│ btrfs        │ Snapshots, RAID       │ CoW, snapshots, checksums, │
│              │ NAS, Btrfs RAID       │ online resize/defrag       │
├──────────────┼──────────────────────┼────────────────────────────┤
│ tmpfs        │ RAM-based temp data   │ Extremely fast, auto-clear │
│              │ /tmp, /run, /dev/shm  │ on reboot                  │
├──────────────┼──────────────────────┼────────────────────────────┤
│ zfs          │ Enterprise storage    │ Checksums, dedup, ARC,     │
│              │ (via OpenZFS)         │ full data integrity        │
└──────────────┴──────────────────────┴────────────────────────────┘
```

🔷 ext4 — The workhorse

```
Design: Extent-based successor to ext3
Default on: Ubuntu, Debian, many RHEL versions

Internal structure:
┌──────────────────────────────────────────────────┐
│  Superblock (filesystem metadata)                 │
│  Block Group 0:                                   │
│    Group Descriptor                               │
│    Block Bitmap   (which blocks are used)         │
│    Inode Bitmap   (which inodes are used)         │
│    Inode Table    (array of inodes)               │
│    Data Blocks    (actual file content)           │
│  Block Group 1... (repeat)                        │
└──────────────────────────────────────────────────┘

Key features:
- Extents: contiguous block ranges (not individual block pointers)
  → Huge performance improvement for large files
- Journaling: writes journal entry first, then data
  → Crash recovery in seconds (not hours like old fsck)
- Delayed allocation: buffers writes, allocates blocks at flush
  → Better space efficiency, larger sequential writes
- HTree directories: hash-tree for large directories
  → Fast lookup even with millions of files
```

```bash
# Create ext4 filesystem
sudo mkfs.ext4 /dev/sdb1
sudo mkfs.ext4 -L "mydata" -m 1 /dev/sdb1
# -L: label
# -m 1: 1% reserved blocks (default 5% — reduce for data disks)

# Inspect ext4 superblock
sudo tune2fs -l /dev/sdb1
```

```
# Filesystem state:          clean
# Reserved block count:      26214   ← the 5% reserved for root
# Last mount time:           Wed Mar 10 2026
# Mount count:               47
# Filesystem features:       extent journaling
# Default mount options:     user_xattr acl

# Adjust settings
sudo tune2fs -m 1 /dev/sdb1        # Reduce reserved to 1%
sudo tune2fs -L "newlabel" /dev/sdb1  # Change label
sudo tune2fs -i 0 /dev/sdb1        # Disable time-based fsck

# Force check on next boot
sudo tune2fs -C 30 /dev/sdb1      # Mark as mounted 30 times (triggers
fsck at 31)

# Check filesystem (must be unmounted)
sudo fsck.ext4 -f /dev/sdb1

# Grow ext4 (online, while mounted)
sudo resize2fs /dev/sdb1          # Grow to fill device
sudo resize2fs /dev/sdb1 50G      # Shrink to 50G (must unmount first for
shrink)

# Strengths: universal, stable, fast recovery
# Weaknesses: max 1EB filesystem, 16TB file; no built-in snapshots; slower
than xfs at scale
```

🔷 xfs — High-performance workhorse

```
Design: B-tree based, designed for parallelism
Default on: RHEL/CentOS/Rocky/Alma 7+, Amazon Linux
Used by: large databases, high-throughput storage, media servers

Internal structure:
┌──────────────────────────────────────────────────────────┐
│  Superblock                                              │
│  Allocation Groups (AG) — multiple, parallel!            │
│  ┌──────────────────────┐  ┌──────────────────────┐      │
│  │  AG 0                │  │  AG 1                │      │
│  │  B-tree: free space  │  │  B-tree: free space  │      │
│  │  B-tree: inodes      │  │  B-tree: inodes      │      │
│  │  Data extents        │  │  Data extents        │      │
│  └──────────────────────┘  └──────────────────────┘      │
│  Multiple AGs = parallel I/O on multiple cores!          │
└──────────────────────────────────────────────────────────┘

Key features:
- Allocation Groups: parallel I/O paths (one per CPU core)
- Delayed logging: groups metadata changes in memory before writing
```

```
journal
- Speculative preallocation: pre-allocates space for growing files
- No fsck on mount: journal replay only (fast recovery)
```

```
# Create xfs filesystem
sudo mkfs.xfs /dev/sdb1
sudo mkfs.xfs -L "mydata" /dev/sdb1

# Inspect xfs
sudo xfs_info /dev/sdb1
# meta-data=/dev/sdb1  isize=512  agcount=4, agsize=32505856 blks
#          =           sectsz=512 attr=2, projid32bit=1
# data     =           bsize=4096 blocks=130023424, imaxpct=25
# realtime =none       extsz=4096 blocks=0, rtextents=0
# agcount=4 means 4 Allocation Groups — 4 parallel I/O paths

# Check and repair xfs (must be unmounted)
sudo xfs_repair /dev/sdb1

# Grow xfs (online, while mounted)
sudo xfs_growfs /mountpoint     # Can only GROW, never shrink

# Defragment xfs (online)
sudo xfs_fsr /mountpoint

# Dump and restore xfs
sudo xfsdump -l 0 -f /backup/dump /data    # Level 0 = full dump
sudo xfsrestore -f /backup/dump /data       # Restore

# Freeze filesystem (for consistent snapshots)
sudo xfs_freeze -f /data       # Freeze (flush + block writes)
# Take snapshot here
sudo xfs_freeze -u /data       # Unfreeze

# Strengths: parallel I/O, huge files (8EB), fast at scale, online grow
# Weaknesses: cannot shrink, worse small-file performance, cannot undelete
```

🔷 btrfs — The modern filesystem

```
Design: Copy-on-Write (CoW), built-in RAID, snapshots
Stable: yes for RAID 0/1/10; RAID 5/6 still has issues
Used by: openSUSE, Fedora (default), NAS systems

Copy-on-Write mechanism:
Write to file:
  OLD:  [Block A] ← file points here
  NEW:  Write data to [Block B] (new location)
        Update file pointer to [Block B]
```

```
        [Block A] becomes free eventually

This enables:
  Snapshots: point to same blocks, diverge on write
  Checksums: each block has checksum in metadata
  Dedup: identical blocks shared automatically
```

```
# Create btrfs
sudo mkfs.btrfs /dev/sdb1
sudo mkfs.btrfs -L "mydata" /dev/sdb1

# Multi-device btrfs (built-in software RAID)
sudo mkfs.btrfs -d raid1 -m raid1 /dev/sdb1 /dev/sdc1   # RAID 1
sudo mkfs.btrfs -d raid0 -m raid0 /dev/sdb1 /dev/sdc1   # RAID 0

# Show btrfs info
sudo btrfs filesystem show /data
sudo btrfs filesystem usage /data

# ── Subvolumes (key btrfs feature) ──────────────────────────────

# Subvolumes are like lightweight partitions within btrfs
sudo btrfs subvolume create /data/myapp
sudo btrfs subvolume list /data

# ── Snapshots (killer feature) ──────────────────────────────────

# Read-only snapshot (instant, nearly zero space)
sudo btrfs subvolume snapshot -r /data/myapp /data/snapshots/myapp-
20260310
# Just metadata — no data copied!
# Space used = only the blocks that differ from original

# Writeable snapshot
sudo btrfs subvolume snapshot /data/myapp /data/myapp-backup

# Rollback: delete current, rename snapshot
sudo btrfs subvolume delete /data/myapp
sudo mv /data/myapp-backup /data/myapp

# ── Scrub — data integrity check ────────────────────────────────

sudo btrfs scrub start /data    # Background checksum verification
sudo btrfs scrub status /data   # Check progress

# ── Balance — redistribute data across devices ──────────────────

sudo btrfs balance start /data
sudo btrfs balance status /data

# Strengths: snapshots, checksums, online resize, built-in RAID,
```

```
subvolumes
# Weaknesses: RAID 5/6 not production-ready, higher CPU overhead
```

🔷 tmpfs — RAM-backed filesystem

```
tmpfs: filesystem that lives entirely in RAM (and swap if needed)
- Disappears completely on reboot
- Extremely fast (memory speed)
- Size is flexible (expands/shrinks as needed, up to configured max)
- Used by kernel for: /dev/shm, /run, /sys/fs/cgroup
```

```
# Mount tmpfs
sudo mount -t tmpfs -o size=512m tmpfs /mnt/ramdisk
sudo mount -t tmpfs -o size=2g,mode=1777 tmpfs /tmp

# In /etc/fstab
tmpfs  /tmp      tmpfs  defaults,size=1g,mode=1777  0 0
tmpfs  /dev/shm  tmpfs  defaults,size=4g            0 0

# Check tmpfs usage
df -h /dev/shm
# Filesystem  Size  Used Avail Use% Mounted on
# tmpfs       7.8G  1.2G  6.6G  16% /dev/shm

# Use cases:
# /tmp        → temp files (cleared on reboot automatically)
# /dev/shm    → POSIX shared memory (databases, IPC)
# Build cache → put build artifacts here for speed
# Browser tmp → Firefox/Chrome profile speedup

# ⚠ tmpfs counts toward RAM usage
# size= limit prevents OOM from runaway tmp usage
# default size = 50% of RAM if not specified

# Production pattern: speed up compilations
sudo mount -t tmpfs -o size=8g tmpfs /tmp/build
cd /tmp/build && cmake /source && make -j$(nproc)
# 3-5x faster than disk-based build
```

🔷 Short crisp interview answer

> "ext4 is my default choice — it's universal, stable, and fast to fsck. xfs is better for large files and high-throughput I/O because of its parallel allocation groups — it's the RHEL default and excellent for databases. btrfs adds Copy-on-Write snapshots, checksums, and built-in RAID but I'd only use RAID 0/1/10, not 5/6. tmpfs is RAM-backed — I use it for `/tmp`, `/dev/shm`, and build caches. The key

> trade-off: ext4 for safety and compatibility, xfs for scale and performance, btrfs for snapshot capability."

⚠ Gotchas

```
# GOTCHA 1: xfs cannot shrink
sudo xfs_growfs /data      # Fine — grows online
# Cannot shrink xfs filesystem at all — plan your size upfront

# GOTCHA 2: btrfs RAID 5/6 has known data loss bugs
# Never use btrfs RAID 5/6 in production for critical data
# Use RAID 1 or RAID 10 with btrfs

# GOTCHA 3: tmpfs uses RAM — not "free" storage
# A full tmpfs = RAM pressure = OOM killer can fire
# Always set size= limits in fstab/mount

# GOTCHA 4: ext4 delayed allocation and crashes
# ext4 buffers writes (delayed allocation)
# If power loss before flush: new files may be zero-length
# Fix: mount with data=ordered (default) or data=journal (slower, safer)

# GOTCHA 5: CoW and database performance
# btrfs CoW is BAD for databases (lots of random small writes)
# Disable CoW for database files:
chattr +C /var/lib/postgresql/   # Disable CoW for postgres data dir
# Or: put databases on xfs instead
```

6.4 `lsblk`, `blkid`, `fdisk`, `parted` — Block Device Management

🔷 What block devices are

A block device is a storage device accessed in fixed-size chunks (blocks). Every disk, partition, RAID array, and LVM logical volume appears as a block device under `/dev/`.

```
Physical disk: /dev/sda          (whole disk)
  ├── /dev/sda1                  (partition 1)
  ├── /dev/sda2                  (partition 2)
  └── /dev/sda3                  (partition 3)

NVMe disk:    /dev/nvme0          (whole disk)
  ├── /dev/nvme0n1               (namespace 1)
  ├── /dev/nvme0n1p1             (partition 1)
  └── /dev/nvme0n1p2             (partition 2)
```

```
LVM:          /dev/mapper/vg0-lv0  (logical volume)
RAID:         /dev/md0              (software RAID device)
```

🔷 `lsblk` — List block devices

```
# Basic tree view
lsblk
# NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
# sda      8:0    0   50G  0 disk
# ├─sda1   8:1    0    1G  0 part /boot
# ├─sda2   8:2    0    4G  0 part [SWAP]
# └─sda3   8:3    0   45G  0 part /
# sdb      8:16   0  500G  0 disk
# └─sdb1   8:17   0  500G  0 part /data
# sr0     11:0    1 1024M  0 rom

# With filesystem info
lsblk -f
# NAME   FSTYPE  LABEL  UUID                                 MOUNTPOINT
# sda
# ├─sda1 ext4           abc123-...                           /boot
# ├─sda2 swap           def456-...                           [SWAP]
# └─sda3 ext4           ghi789-...                           /
# sdb
# └─sdb1 xfs    mydata  jkl012-...                           /data

# All info (size, type, UUID, label, mountpoint)
lsblk -o NAME,SIZE,FSTYPE,UUID,LABEL,MOUNTPOINT,TYPE

# Show disk model and serial (useful for physical identification)
lsblk -d -o NAME,MODEL,SERIAL,SIZE,ROTA
# ROTA=1 means rotational (HDD), ROTA=0 means SSD/NVMe

# Show only disks (no partitions)
lsblk -d
```

🔷 `blkid` — Block device identification

```
# Show all block devices with UUID and filesystem type
sudo blkid
# /dev/sda1: UUID="abc123" TYPE="ext4" PARTUUID="xyz789"
# /dev/sda2: UUID="def456" TYPE="swap"
# /dev/sdb1: UUID="jkl012" TYPE="xfs" LABEL="mydata"

# Specific device
sudo blkid /dev/sdb1
```

```
# Just the UUID (for scripting / fstab)
sudo blkid -s UUID -o value /dev/sdb1
# jkl012-4a1b-...

# Just the type
sudo blkid -s TYPE -o value /dev/sdb1
# xfs

# Find device by UUID
sudo blkid -U "jkl012-4a1b-..."
# /dev/sdb1

# Production use: generate fstab entry
UUID=$(sudo blkid -s UUID -o value /dev/sdb1)
TYPE=$(sudo blkid -s TYPE -o value /dev/sdb1)
echo "UUID=$UUID  /data  $TYPE  defaults,noatime  0  2" | sudo tee -a
/etc/fstab
```

🔷 `fdisk` — MBR partition management

```
# ── View partition table ──────────────────────────────────────

# List partitions on all disks
sudo fdisk -l

# List specific disk
sudo fdisk -l /dev/sdb
# Disk /dev/sdb: 500 GiB
# Disk model: Samsung SSD 870
# Disklabel type: gpt
# Device      Start        End   Sectors  Size Type
# /dev/sdb1    2048    2099199   2097152    1G Linux filesystem
# /dev/sdb2 2099200 1048575966 1046476767  499G Linux filesystem

# ── Interactive partition editing ─────────────────────────────
sudo fdisk /dev/sdb

# fdisk commands:
# p → print current partition table
# n → new partition
# d → delete partition
# t → change partition type
# l → list partition types
# g → create new GPT partition table
# o → create new MBR partition table
# w → write changes (commits!)
# q → quit WITHOUT saving

# Example workflow:
```

```
sudo fdisk /dev/sdb
# Command: g          ← create GPT partition table
# Command: n          ← new partition
# Partition number: 1
# First sector: 2048  ← accept default
# Last sector: +100G  ← 100GB partition
# Command: n          ← second partition
# Partition number: 2
# First sector: (accept default — after partition 1)
# Last sector: (accept default — rest of disk)
# Command: w          ← write and exit

# After creating partition, create filesystem:
sudo mkfs.xfs /dev/sdb1
sudo mkfs.ext4 /dev/sdb2

# ⚠ fdisk is for MBR (up to 2TB) and GPT
# For disks > 2TB: must use GPT (fdisk supports GPT too)
# For simpler GPT workflow: use parted or gdisk
```

🔷 `parted` — Advanced partition management

```
# ── Non-interactive (scriptable) ──────────────────────────────

# Print partition table
sudo parted /dev/sdb print

# Create GPT partition table
sudo parted /dev/sdb mklabel gpt

# Create partition (parted uses MB/GB directly)
sudo parted /dev/sdb mkpart primary xfs 1MiB 100GiB
sudo parted /dev/sdb mkpart primary xfs 100GiB 100%

# Set partition flags
sudo parted /dev/sdb set 1 boot on    # Boot flag
sudo parted /dev/sdb set 2 lvm on     # LVM flag

# Resize partition (parted can do this)
sudo parted /dev/sdb resizepart 2 200GiB

# ── Interactive mode ──────────────────────────────────────────
sudo parted /dev/sdb

# (parted) print        ← view partitions
# (parted) mklabel gpt  ← GPT partition table
# (parted) mkpart primary xfs 1MiB 100GiB
# (parted) mkpart primary xfs 100GiB 100%
# (parted) quit
```

```
# ── Inform kernel of partition changes ──────────────────────────────

# After changing partitions on a MOUNTED disk:
sudo partprobe /dev/sdb    # Ask kernel to re-read partition table
# Or
sudo blockdev --rereadpt /dev/sdb

# ── Full disk provisioning workflow ─────────────────────────────────

DISK=/dev/sdb

# 1. Create GPT partition table
sudo parted "$DISK" mklabel gpt

# 2. Create single partition spanning whole disk
sudo parted "$DISK" mkpart primary xfs 1MiB 100%

# 3. Set LVM flag (if using LVM)
sudo parted "$DISK" set 1 lvm on

# 4. Inform kernel
sudo partprobe "$DISK"

# 5. Create filesystem
sudo mkfs.xfs "${DISK}1"

# 6. Create mount point and mount
sudo mkdir -p /data
sudo mount "${DISK}1" /data

# 7. Add to fstab
UUID=$(sudo blkid -s UUID -o value "${DISK}1")
echo "UUID=$UUID  /data  xfs  defaults,noatime  0  2" | sudo tee -a
/etc/fstab
```

🔷 Short crisp interview answer

> "`lsblk` gives me the tree view of all block devices with mount points. `blkid` gives UUIDs and filesystem types — I always use UUIDs in `/etc/fstab` instead of `/dev/sdX` names because device names can change on reboot. For partitioning, I prefer `parted` over `fdisk` for scripting because it's non-interactive and supports both MBR and GPT. After any partition changes on a live disk, `partprobe /dev/sdX` tells the kernel to re-read the partition table without rebooting."

6.5 LVM — Physical Volumes, Volume Groups, Logical Volumes

🔷 What LVM is and why it exists

LVM (**Logical Volume Manager**) adds a **virtualization layer** between physical disks and filesystems. Without LVM, a filesystem is locked to one partition on one disk. With LVM, you can resize volumes on the fly, span multiple disks, take snapshots, and add storage without downtime.

🔷 LVM architecture

```
Physical World:                LVM Layer:                Filesystem Layer:

                               ┌─────────────────┐
/dev/sda1 (PV) ──────────────►│                 │──────────► /dev/vg0/lv_root
→ ext4 → /
/dev/sdb   (PV) ─────────────► Volume Group    │──────────► /dev/vg0/lv_data
→ xfs  → /data
/dev/sdc   (PV) ─────────────►│  (vg0)          │──────────► /dev/vg0/lv_swap
→ swap
                               │                 │
                               └─────────────────┘


Three layers:
PV (Physical Volume)  → raw disk or partition prepared for LVM
VG (Volume Group)     → pool of PV storage combined
LV (Logical Volume)   → slice of VG, appears as block device, gets
filesystem

PE (Physical Extent)  → smallest unit of allocation in VG (default 4MB)
LE (Logical Extent)   → corresponds to PE inside an LV
```

🔷 Working with LVM

```
# ── Physical Volumes ──────────────────────────────────────────────

# Prepare a disk/partition as PV
sudo pvcreate /dev/sdb
sudo pvcreate /dev/sdc
sudo pvcreate /dev/sdd1    # Can use whole disk or partition

# View PVs
sudo pvs
# PV         VG   Fmt  Attr PSize   PFree
# /dev/sdb   vg0  lvm2 a--  500.00g    0
# /dev/sdc   vg0  lvm2 a--  500.00g  200g
# /dev/sdd1       lvm2 ---  200.00g  200g  ← not yet in a VG

sudo pvdisplay /dev/sdb    # Detailed info
sudo pvdisplay             # All PVs detailed

# ── Volume Groups ─────────────────────────────────────────────────

# Create VG from PVs
```

```
sudo vgcreate vg0 /dev/sdb /dev/sdc

# Add a PV to existing VG (extend the pool!)
sudo vgextend vg0 /dev/sdd1

# View VGs
sudo vgs
# VG   #PV #LV #SN Attr   VSize    VFree
# vg0    3   2   0 wz--n- 1200.00g 400.00g

sudo vgdisplay vg0

# Rename VG
sudo vgrename vg0 vg_production

# ── Logical Volumes ──────────────────────────────────────────

# Create LV
sudo lvcreate -L 100G -n lv_data vg0        # Fixed size (100GB)
sudo lvcreate -l 100%FREE -n lv_data vg0    # Use all free space
sudo lvcreate -l 50%VG -n lv_data vg0       # 50% of VG

# View LVs
sudo lvs
# LV      VG   Attr       LSize   Pool Origin Data%
# lv_root vg0  -wi-ao----  50.00g
# lv_data vg0  -wi-ao---- 100.00g

sudo lvdisplay /dev/vg0/lv_data   # Detailed

# LV path conventions:
# /dev/vg0/lv_data          ← symlink
# /dev/mapper/vg0-lv_data   ← actual device mapper path

# Create filesystem on LV (just like any block device)
sudo mkfs.xfs /dev/vg0/lv_data
sudo mount /dev/vg0/lv_data /data

# ── Extending (online!) ──────────────────────────────────────

# Extend LV by 50GB
sudo lvextend -L +50G /dev/vg0/lv_data

# Extend LV to use all free space in VG
sudo lvextend -l +100%FREE /dev/vg0/lv_data

# Extend filesystem to match (ext4)
sudo resize2fs /dev/vg0/lv_data

# Extend filesystem to match (xfs — online!)
sudo xfs_growfs /data

# One-liner: extend LV AND resize filesystem together
sudo lvextend -L +50G -r /dev/vg0/lv_data
```

```
# -r = resize filesystem automatically (handles ext4 and xfs)

# ── Reducing (offline only, ext4 only!) ──────────────────────────

# ⚠ Cannot shrink xfs — only ext4!
# Must unmount first

sudo umount /data
sudo e2fsck -f /dev/vg0/lv_data        # Must check first
sudo resize2fs /dev/vg0/lv_data 80G    # Shrink filesystem to 80G
sudo lvreduce -L 80G /dev/vg0/lv_data  # Shrink LV to match
sudo mount /dev/vg0/lv_data /data

# ── LVM Snapshots ────────────────────────────────────────────────

# Create snapshot (point-in-time copy)
sudo lvcreate -L 10G -s -n lv_data_snap /dev/vg0/lv_data
# -s = snapshot
# -L 10G = snapshot COW buffer size (space for changed blocks)
# Snapshot is instant — no data copied yet

# Mount snapshot (read-only, see data as it was at snapshot time)
sudo mount -o ro /dev/vg0/lv_data_snap /mnt/snap

# Backup from snapshot (consistent, original LV still serving traffic)
sudo tar -czf /backup/data.tar.gz -C /mnt/snap .

# Restore from snapshot (merge back)
sudo umount /data
sudo umount /mnt/snap
sudo lvconvert --merge /dev/vg0/lv_data_snap
sudo mount /dev/vg0/lv_data /data

# Delete snapshot
sudo lvremove /dev/vg0/lv_data_snap

# ⚠ Monitor snapshot usage — if COW buffer fills up, snapshot is
invalidated!
sudo lvs -o lv_name,data_percent
# lv_data_snap   45.3   ← 45% of snapshot space used — getting close!

# ── Moving PVs (for disk replacement) ────────────────────────────

# Move data off a failing disk
sudo pvmove /dev/sdb           # Move all extents to other PVs in VG
sudo pvmove /dev/sdb /dev/sdd  # Move to specific PV

# Then safely remove the disk
sudo vgreduce vg0 /dev/sdb     # Remove PV from VG
sudo pvremove /dev/sdb         # Remove PV label
# Now physical disk can be unplugged!
```

🔷 Short crisp interview answer

> "LVM adds a virtualization layer between physical disks and filesystems with three layers: Physical Volumes (PVs) are prepared disks, Volume Groups (VGs) pool PVs together, and Logical Volumes (LVs) are carved from VGs and get filesystems. The power is flexibility: I can extend an LV online with `lvextend -L +50G -r /dev/vg0/lv_data` which resizes both the LV and filesystem in one command. I can add disks to a VG with `vgextend`, take snapshots with `lvcreate -s`, and replace failing disks with `pvmove` — all without downtime."

⚠ Gotchas

```
# GOTCHA 1: Snapshot invalidation
# If snapshot COW buffer fills → snapshot becomes invalid (UNUSABLE)
# Monitor: sudo lvs -o lv_name,data_percent
# Size snapshot buffer at least 15-20% of source LV for safety

# GOTCHA 2: xfs cannot shrink
# Never put xfs on an LV you might need to shrink
# Use ext4 if you need shrink capability

# GOTCHA 3: pvmove is slow and risky on large disks
# pvmove moves extents — on a 2TB disk, can take hours
# If interrupted: sudo pvmove --abort, then retry
# Power loss during pvmove: metadata may be inconsistent

# GOTCHA 4: LV path vs device mapper path
ls /dev/vg0/lv_data          # Symlink — OK
ls /dev/mapper/vg0-lv_data   # Real device — note: dash not slash
# Both work, but device mapper path is canonical

# GOTCHA 5: VG metadata backup
# LVM auto-backs up metadata to /etc/lvm/backup/
# After disk replacement or VG changes: vgcfgbackup vg0
```

6.6 `iostat`, `iotop` — I/O Performance Analysis

🔷 What they are

`iostat` (**I/O stat**istics) reports CPU and disk I/O statistics from the kernel. `iotop` shows which processes are generating I/O — the `top` equivalent for disk activity.

🔷 How Linux I/O works (the path a write takes)

```
Application write()
       │
       ▼
  Page Cache (RAM)  ◄── Most reads served from here
       │                Writes buffered here (async by default)
       │
  Block Layer
       │
  I/O Scheduler  ◄── Reorders/merges requests for efficiency
       │
  Device Driver
       │
  Physical Disk / NVMe / Network Storage
       │
  Returns to application (async: immediately; sync: after write completes)

iostat measures I/O at the Block Layer level
iotop measures which processes submitted I/O to the Block Layer
```

🔷 `iostat` — Disk I/O statistics

```
# Install: apt install sysstat

# ── Basic usage ──────────────────────────────────────────────

# One-time snapshot (CPU + disk)
iostat

# Human readable + extended disk stats
iostat -h -x

# Continuous monitoring: report every 2 seconds
iostat -h -x 2

# Specific device only
iostat -h -x /dev/sda 2

# ── Output explained ─────────────────────────────────────────

iostat -hx 2
# avg-cpu:  %user  %nice  %system  %iowait  %steal   %idle
#            2.5%   0.0%     1.3%     8.7%    0.0%   87.5%
#                                   ────
#                             High iowait = waiting on disk I/O

# Device:         r/s    w/s    rkB/s    wkB/s  rrqm/s  wrqm/s  %rrqm
%wrqm  r_await  w_await  aqu-sz  rareq-sz  wareq-sz   %util
# sda            15.2   42.8    480.0   1712.0     0.5     3.2    3.2
7.0      2.3     12.4    0.58     31.6     40.0      13.2
```

```
# sdb            0.0  198.4      0.0   7936.0     0.0    22.4    0.0
11.3      0.0      1.2    0.24      0.0     40.0      23.8

# Column meanings — the critical ones:
# r/s       → reads per second
# w/s       → writes per second
# rkB/s     → KB read per second
# wkB/s     → KB written per second
# r_await   → avg time (ms) for reads to complete (latency!)
# w_await   → avg time (ms) for writes to complete (latency!)
# aqu-sz    → avg queue depth (how many I/Os waiting)
# %util     → how busy the device is (100% = saturated)

# ── Key metrics to watch ──────────────────────────────────────

# %util:
# < 80%   → disk has headroom
# 80-90%  → getting busy, monitor closely
# > 95%   → disk is saturated — I/O bottleneck!
# 100%    → disk fully saturated, requests queuing

# await (latency):
# HDD:   normal r_await < 10ms, w_await < 5ms (with cache)
# SSD:   normal < 1ms
# NVMe:  normal < 0.1ms
# High latency = disk under heavy load or failing

# aqu-sz (queue depth):
# > 1 consistently = disk cannot keep up with requests

# ── Practical patterns ────────────────────────────────────────

# Monitor all disks, update every 1 second, only show disk stats
iostat -hxd 1

# Show which disk is busiest
iostat -hx 1 | awk '/^[a-z]/ && $NF+0 > 50 {print $0}'

# Save to file for later analysis
iostat -hx 1 > /tmp/iostat_$(date +%Y%m%d_%H%M).log &

# CPU iowait spike investigation
iostat -hx 5 | grep -E "^(avg-cpu|Device|sd|nvme)" | head -20
```

🔷 `iotop` — Per-process I/O monitoring

```
# Install: apt install iotop

# Interactive mode (like top but for I/O)
sudo iotop
```

```
# Output:
# Total DISK READ: 25.2 M/s   |  Total DISK WRITE: 102.4 M/s
# TID   PRIO  USER  DISK READ  DISK WRITE  SWAPIN    IO>   COMMAND
# 1234  be/4  mysql    0.00 B/s   98.3 M/s  0.00 %  94.5 % mysqld
# 5678  be/4  root   25.2 M/s     0.00 B/s  0.00 %  89.2 % rsync
# 9012  be/4  www      0.00 B/s    3.5 M/s  0.00 %   2.1 % apache2

# Column meanings:
# DISK READ/WRITE → actual I/O rates for this process
# SWAPIN          → reading from swap (memory pressure indicator!)
# IO>             → % of time this process spent waiting for I/O

# ── Non-interactive mode ──────────────────────────────────────

# Run once and exit
sudo iotop -b -n 3     # Batch mode, 3 iterations
sudo iotop -b -n 1 -o  # -o = only show processes doing I/O

# Show only processes with active I/O
sudo iotop -o

# Specific process
sudo iotop -p 1234

# ── Combining iostat + iotop ──────────────────────────────────

# Workflow for I/O investigation:
# 1. iostat shows WHICH disk is saturated
iostat -hx 1

# 2. iotop shows WHICH process is causing it
sudo iotop -o

# 3. lsof shows WHICH files that process is accessing
sudo lsof -p 1234 | grep -E "REG|DIR"

# 4. strace shows WHAT I/O syscalls it's making
sudo strace -p 1234 -e trace=read,write,pread64,pwrite64

# ── Other useful I/O tools ────────────────────────────────────

# dstat — combines cpu, disk, net, memory in one view
dstat -tdD total,sda,sdb 5

# pidstat — per-process I/O stats (from sysstat package)
pidstat -d 2           # I/O stats every 2 seconds
pidstat -p 1234 -d 1  # Specific process

# /proc/diskstats — raw kernel disk stats
cat /proc/diskstats
# Parsed by iostat — useful for scripted monitoring

# Block device queue stats
```

```
cat /sys/block/sda/queue/scheduler    # Current I/O scheduler
cat /sys/block/sda/stat               # Device stats (reads, writes, etc.)
```

🔷 Short crisp interview answer

> "`iostat -hx 2` is my first tool for disk performance — the key metrics are `%util` (how saturated the disk is, bad above 95%), `r_await`/`w_await` (I/O latency — should be <1ms for SSD, <10ms for HDD), and `aqu-sz` (queue depth — consistently >1 means disk can't keep up). When `iostat` shows a saturated disk, I use `sudo iotop -o` to see which process is generating the I/O, then `lsof -p <PID>` to see which files it's accessing. High `%iowait` in `iostat` CPU section means CPUs are idle waiting for disk — classic I/O bottleneck."

⚠ Gotchas

```
# GOTCHA 1: %util=100% doesn't mean disk is fully used for SSDs/NVMe
# SSDs can process multiple I/Os in parallel (high queue depth)
# %util only shows "was the device busy?" not "is it saturated?"
# For SSDs: watch aqu-sz and latency — more reliable saturation indicators

# GOTCHA 2: %iowait is system-wide, not per-disk
# High iowait doesn't tell you WHICH disk — use iostat device columns

# GOTCHA 3: iostat first report is since-boot average
# First output of iostat = averages since boot (not useful for current
state)
# Always use: iostat -x 2  (skip first report, watch subsequent ones)

# GOTCHA 4: iotop requires root
sudo iotop    # Works
iotop         # Permission denied or shows 0 for other users' processes

# GOTCHA 5: Buffered vs Direct I/O
# iotop shows ACTUAL I/O to disk, not application write() calls
# Application may buffer 100MB in memory, then flush → iotop shows the
flush
# This makes iotop I/O appear bursty even for steady-write applications
```

6.7 RAID Concepts — RAID 0/1/5/6/10

🔷 What RAID is

RAID (**R**edundant **A**rray of **I**nexpensive **D**isks) combines multiple physical disks into a single logical unit to improve **performance**, **redundancy**, or both. Linux supports software RAID via `mdadm`.

🔷 RAID levels visual overview

```
RAID 0 — Striping (performance, NO redundancy)

  ┌────┐ ┌────┐
  │ A1 │ │ A2 │     File A is split: A1 on disk1, A2 on disk2
  │ B1 │ │ B2 │     Read/write speed = 2× (reads both in parallel)
  └────┘ └────┘     ONE disk fails → ALL data lost
 Disk1  Disk2

RAID 1 — Mirroring (redundancy, not performance)

  ┌────┐ ┌────┐
  │ A  │ │ A  │     File A is written to BOTH disks identically
  │ B  │ │ B  │     Read speed = 2× (can read from either)
  └────┘ └────┘     Write speed = 1× (must write to both)
 Disk1  Disk2       ONE disk can fail — survive with 0 data loss

RAID 5 — Striping + Distributed Parity (balance)

  ┌────┐ ┌────┐ ┌────┐
  │ A1 │ │ A2 │ │ Ap │    Ap = parity of A1,A2 (can reconstruct either)
  │ B1 │ │ Bp │ │ B2 │    Parity rotates across disks
  │ Cp │ │ C1 │ │ C2 │
  └────┘ └────┘ └────┘
 Disk1  Disk2  Disk3      Capacity = (N-1) × disk_size
                          ONE disk can fail — survive

RAID 6 — Striping + Double Parity

  ┌────┐ ┌────┐ ┌────┐ ┌────┐
  │ A1 │ │ A2 │ │ Ap │ │ Aq │    Two independent parity blocks
  │ B1 │ │ Bp │ │ Bq │ │ B2 │
  └────┘ └────┘ └────┘ └────┘
 Disk1  Disk2  Disk3  Disk4     Capacity = (N-2) × disk_size
                                TWO disks can fail simultaneously

RAID 10 — Mirrored Stripes (best of RAID 1 + RAID 0)

  ┌────┐ ┌────┐   ┌────┐ ┌────┐
  │ A1 │ │ A1 │   │ A2 │ │ A2 │   Stripe across mirrors
  │ B1 │ │ B1 │   │ B2 │ │ B2 │
  └────┘ └────┘   └────┘ └────┘
 Mirror pair 1   Mirror pair 2
 Read speed ≈ 2× write speed, ONE disk per mirror can fail
 Capacity = 50% of total raw
```

🔷 RAID comparison table

```
┌──────────┬──────────┬──────────────┬──────────┬──────────────────────────────
──┐
│ Level    │ Min Disks│ Fault Tol.   │ Capacity │ Best Use Case
│
├──────────┼──────────┼──────────────┼──────────┼──────────────────────────────
```

```
──┤
│ RAID 0 │    2    │ None         │ 100%     │ Scratch space, where speed
│
│        │         │              │          │ matters and data is
ephemeral│
├────────┼─────────┼──────────────┼──────────┼──────────────────────────
──┤
│ RAID 1 │    2    │ 1 disk fail  │ 50%      │ OS disks, small critical
data│
├────────┼─────────┼──────────────┼──────────┼──────────────────────────
──┤
│ RAID 5 │    3    │ 1 disk fail  │ (N-1)/N  │ General storage, cost-eff.
│
│        │         │              │          │ ⚠ Vulnerable during rebuild
│
├────────┼─────────┼──────────────┼──────────┼──────────────────────────
──┤
│ RAID 6 │    4    │ 2 disk fail  │ (N-2)/N  │ Large arrays, slow HDDs
│
├────────┼─────────┼──────────────┼──────────┼──────────────────────────
──┤
│RAID 10 │    4    │ 1 per mirror │ 50%      │ Databases, high I/O
critical │
└────────┴─────────┴──────────────┴──────────┴──────────────────────────
──┘
```

🔷 Linux software RAID with `mdadm`

```
# ── Create RAID arrays ──────────────────────────────────────────

# RAID 0 (striping)
sudo mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/sdb /dev/sdc

# RAID 1 (mirroring)
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc

# RAID 5 (striping + parity)
sudo mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb /dev/sdc
/dev/sdd

# RAID 6 (double parity)
sudo mdadm --create /dev/md0 --level=6 --raid-devices=4 /dev/sdb /dev/sdc
/dev/sdd /dev/sde

# RAID 10 (mirrored stripes)
sudo mdadm --create /dev/md0 --level=10 --raid-devices=4 /dev/sdb /dev/sdc
/dev/sdd /dev/sde

# After creation: create filesystem and mount
sudo mkfs.xfs /dev/md0
```

```
sudo mount /dev/md0 /data

# ── Monitor array status ──────────────────────────────────────

# Quick status
sudo mdadm --detail /dev/md0
# /dev/md0:
#   Raid Level : raid5
#   Array Size : 999G
#  Used Dev Size: 500G
#  Raid Devices : 3
# Active Devices: 3
#   State : clean   ← GOOD: clean = healthy, degraded = disk failed!
# Number   Major   Minor   RaidDevice  State
#    0       8       16        0      active sync   /dev/sdb
#    1       8       32        1      active sync   /dev/sdc
#    2       8       48        2      active sync   /dev/sdd

# Short status (all arrays)
cat /proc/mdstat
# Personalities : [raid5] [raid6]
# md0 : active raid5 sdd[2] sdc[1] sdb[0]
#       999G super 1.2 512K chunks 2 near-copies
#       [3/3] [UUU]   ← [3/3] = 3 of 3 disks active, U=up F=failed

# Monitor continuously
watch -n 5 cat /proc/mdstat

# ── Simulating and handling disk failure ──────────────────────

# Mark a disk as failed (testing)
sudo mdadm /dev/md0 --fail /dev/sdb

# Remove failed disk from array
sudo mdadm /dev/md0 --remove /dev/sdb

# Add replacement disk (rebuild begins automatically)
sudo mdadm /dev/md0 --add /dev/sde

# Watch rebuild progress
watch cat /proc/mdstat
# md0 : active raid5 sde[3] sdc[1] sdd[2]
#       [3/2] [_UU]        ← _ = rebuilding this slot
#       rebuild status 23% (234567/999999) finish=42.5min
speed=200000K/sec

# ── Save RAID configuration ───────────────────────────────────

# Save config so array auto-assembles on boot
sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
sudo update-initramfs -u   # Update initramfs to include mdadm config

# ── Add spare disk ────────────────────────────────────────────
```

```
# Hot spare: automatically used if another disk fails
sudo mdadm /dev/md0 --add /dev/sdf    # Detected as spare automatically
# When disk fails → spare kicks in → rebuild starts immediately

# ── Email alerts for disk failure ──────────────────────────────────
# /etc/mdadm/mdadm.conf
# MAILADDR admin@example.com
# MAILFROM mdadm@server.example.com
sudo systemctl enable mdmonitor
```

🔷 Short crisp interview answer

> "RAID 0 stripes for speed but has zero redundancy — one disk fails, everything is lost. RAID 1 mirrors — any single disk can fail, reads are faster. RAID 5 uses distributed parity across N-1 disks — one failure tolerated, but during rebuild (which can take hours on large disks) a second failure loses everything. RAID 6 uses double parity for two-disk fault tolerance. RAID 10 is my recommendation for databases: it combines mirroring and striping for both performance and redundancy. On Linux, `mdadm` manages software RAID; I watch `/proc/mdstat` for array health. RAID is NOT a backup — it protects against disk failure, not data corruption or deletion."

⚠ Gotchas

```
# GOTCHA 1: RAID is NOT a backup
# RAID 1 mirrors bit-for-bit — if you delete a file, it's deleted from
both
# A virus, rm -rf, or corruption is reflected to all mirrors
# Always have backups independent of RAID

# GOTCHA 2: RAID 5 rebuild danger (URE risk)
# During RAID 5 rebuild: if ANY other disk has an unrecoverable read error
(URE)
# → entire array is LOST
# URE probability: ~1 in 10^14 bits for modern HDDs
# 4TB disk = 3.2×10^13 bits → significant URE probability during rebuild!
# Use RAID 6 for arrays of large HDDs

# GOTCHA 3: Write hole in RAID 5
# Power loss during write: parity may be inconsistent with data
# Fix: mdadm journal (write-intent bitmap) or use hardware RAID with
battery backup

# GOTCHA 4: Software RAID vs hardware RAID
# Software RAID (mdadm): portable — array survives controller change
# Hardware RAID: array is tied to controller — controller fails = data may
be inaccessible
# With modern CPUs, software RAID performance is comparable to hardware
RAID
```

```
# GOTCHA 5: Mismatched disk sizes in RAID
# Array uses the smallest disk's size for all disks
# 4×4TB + 1×2TB RAID 5 = 4×2TB = 8TB usable (not 16TB!)
# Always use same-size disks in RAID
```

6.8 I/O Schedulers — `cfq`, `deadline`, `noop`, `mq-deadline`

🔷 What an I/O scheduler does

When multiple processes request disk I/O simultaneously, the I/O scheduler **reorders and merges requests** before sending them to the disk. The goal: maximize throughput and minimize latency based on the disk type and workload.

🔷 Why reordering matters — the elevator analogy

```
Without scheduler (naive order):
  Read cylinder 1000 → Read cylinder 1 → Read cylinder 999 → ...
  Disk head: 1000 → 1 → 999 → ...  ← huge seeks! Very slow for HDDs

With scheduler (elevator algorithm):
  Sort: Read cylinder 1 → Read cylinder 999 → Read cylinder 1000
  Disk head: 1 → 999 → 1000 → ...  ← sequential sweep, fast!

For SSDs: seek time is ~0 — reordering less important
          Focus shifts to: fairness and latency
```

🔷 The schedulers

```bash
# ── Check current scheduler ──────────────────────────────────────

cat /sys/block/sda/queue/scheduler
# [mq-deadline] kyber bfq none
# ← [] brackets show the active scheduler

# All devices
for disk in /sys/block/*/queue/scheduler; do
    echo "$disk: $(cat $disk)"
done

# ── Available schedulers ─────────────────────────────────────────
```

`noop` / `none` — No reordering

```
# What: Just merges adjacent requests, no reordering
# Who: SSDs, NVMe, virtual disks (in VMs)
# Why: SSDs have no seek penalty — reordering adds overhead with no
benefit
#      Hypervisor below often has its own scheduler — don't double-
schedule

# Set:
echo noop > /sys/block/sda/queue/scheduler      # older kernels
echo none > /sys/block/nvme0n1/queue/scheduler  # newer kernels
```

`deadline` / `mq-deadline` — Latency-first

```
# What: Maintains TWO queues (read and write) with deadlines
#       Requests expire after deadline → guaranteed maximum latency
# Who: Databases (low-latency random I/O is critical)
#      Any latency-sensitive workload
# Why: Prevents starvation — no request waits more than deadline
#      Read deadline: 500ms (default), Write deadline: 5000ms

cat /sys/block/sda/queue/iosched/read_expire   # 500 ms
cat /sys/block/sda/queue/iosched/write_expire  # 5000 ms

# Set:
echo mq-deadline > /sys/block/sda/queue/scheduler

# Tune read deadline (for very latency-sensitive DB)
echo 100 > /sys/block/sda/queue/iosched/read_expire
```

`cfq` — Completely Fair Queuing (legacy)

```
# What: Each process gets its own queue, time-sliced fairly
# Who: Multi-user desktop systems, mixed workloads
# Why: Fairness — no single process can monopolize I/O
# Note: Deprecated in Linux 5.0+, replaced by BFQ

# Time slice per process: 100ms by default
cat /sys/block/sda/queue/iosched/quantum
```

`bfq` — Budget Fair Queuing (modern cfq replacement)

```
# What: Assigns I/O "budgets" to processes — interactive tasks get
priority
```

```
# Who: Desktop systems, mixed workloads needing good responsiveness
# Why: Better interactivity than cfq — UI stays responsive during heavy
I/O

echo bfq > /sys/block/sda/queue/scheduler
```

`kyber` — Multi-queue for fast storage

```
# What: Simple scheduler for NVMe with separate queues per latency class
# Who: High-speed NVMe with low-latency requirements
# Target latency: read=2ms, write=10ms (tunable)

echo kyber > /sys/block/nvme0n1/queue/scheduler
cat /sys/block/nvme0n1/queue/iosched/read_lat_nsec   # 2000000 (2ms)
```

🔷 Recommended scheduler by device type

```
# HDD (rotational):
echo mq-deadline > /sys/block/sda/queue/scheduler   # Database servers
echo bfq > /sys/block/sda/queue/scheduler            # General / desktop

# SSD (non-rotational):
echo mq-deadline > /sys/block/sdb/queue/scheduler   # Database on SSD
echo none > /sys/block/sdb/queue/scheduler           # Generally fine

# NVMe:
echo none > /sys/block/nvme0n1/queue/scheduler       # NVMe is fast enough
echo kyber > /sys/block/nvme0n1/queue/scheduler      # If latency matters

# Virtual disks (in VM/cloud):
echo none > /sys/block/vda/queue/scheduler           # Host has its own
scheduler

# ── Make persistent (survives reboot) ──────────────────────────────

# udev rule approach (recommended):
cat /etc/udev/rules.d/60-ioschedulers.rules
# ACTION=="add|change", KERNEL=="sda", ATTR{queue/scheduler}="mq-deadline"
# ACTION=="add|change", KERNEL=="nvme*", ATTR{queue/scheduler}="none"
# ACTION=="add|change", SUBSYSTEM=="block", ATTR{queue/rotational}=="0",
#   ATTR{queue/scheduler}="mq-deadline"

# Apply without reboot:
sudo udevadm trigger --subsystem-match=block --action=add

# ── Check if disk is rotational ────────────────────────────────────
cat /sys/block/sda/queue/rotational
# 1 = HDD (rotational)
```

```
# 0 = SSD/NVMe (non-rotational)

# Auto-tune script:
for disk in /sys/block/sd* /sys/block/nvme*; do
    name=$(basename "$disk")
    rotational=$(cat "$disk/queue/rotational" 2>/dev/null)
    if [[ "$rotational" == "0" ]]; then
        echo "none" > "$disk/queue/scheduler"
        echo "$name (SSD): set to none"
    else
        echo "mq-deadline" > "$disk/queue/scheduler"
        echo "$name (HDD): set to mq-deadline"
    fi
done
```

🔷 Short crisp interview answer

> "I/O schedulers reorder disk requests to improve throughput and fairness. For HDDs, reordering matters because seek time is high — `mq-deadline` prevents starvation by enforcing maximum wait times, good for databases. For SSDs and NVMe, seek time is near-zero so `none` (no reordering) is usually best — avoid double-scheduling overhead since cloud VMs already have a scheduler at the hypervisor layer. I check `/sys/block/sda/queue/rotational` to determine disk type and `/sys/block/sda/queue/scheduler` to see and set the current scheduler. Changes are persisted via udev rules."

6.9 Page Cache & Buffer Cache — How Linux Uses RAM for I/O

🔷 What they are in simple terms

Linux is aggressive about using free RAM to cache disk data. The **page cache** stores file contents (data pages). The **buffer cache** stores block device metadata (directory entries, inode tables, etc.). Together they make disk I/O feel much faster — a cache hit serves data at RAM speed instead of disk speed.

🔷 How the page cache works

```
Read path (first access):
  Application: read("file.txt")
  └→ VFS checks page cache → MISS
      └→ Kernel issues disk read
          └→ Data loaded into page cache (RAM)
              └→ Data returned to application

Read path (subsequent access):
```

```
  Application: read("file.txt")
  └─► VFS checks page cache → HIT
      └─► Data returned from RAM (no disk I/O!)
          → Disk-speed I/O becomes RAM-speed I/O

Write path (default: writeback mode):
  Application: write("file.txt", data)
  └─► Data written to page cache (RAM) → returns immediately to app
      └─► Kernel flusher thread (kswapd/pdflush)
          writes dirty pages to disk periodically
          (every 5 seconds by default, or when memory pressure)

Write path (with O_SYNC or fsync()):
  Application: fsync("file.txt")
  └─► Kernel immediately writes all dirty pages for this file
      └─► Returns only after disk confirms write
          → Slower but guaranteed durable
```

🔷 Reading memory usage

```
# ── free — the standard view ──────────────────────────────────────

free -h
#               total        used        free      shared  buff/cache
available
# Mem:          15.4G        4.2G        1.1G      512.0M       10.1G
10.4G
#               _____        ____        ____                    _____
_____
#               total RAM    used by     truly      shared       page
available
#                            apps        free       tmpfs        cache
to apps

# KEY: "available" is what matters, not "free"!
# available = free + reclaimable cache (cache Linux will drop under
pressure)
# Linux keeps RAM full of cache — "free" being low is NORMAL AND GOOD

# ── /proc/meminfo — detailed breakdown ────────────────────────────

cat /proc/meminfo
# MemTotal:      16148936 kB   ← total physical RAM
# MemFree:        1124820 kB   ← truly unused RAM
# MemAvailable:  10643916 kB   ← available to apps (free + reclaimable)
# Buffers:          98524 kB   ← buffer cache (block metadata)
# Cached:         9876234 kB   ← page cache (file data)
# SwapCached:           0 kB   ← swap also in RAM (double-mapped)
# Active:         5234876 kB   ← recently used pages
# Inactive:       7654320 kB   ← older pages — first to be reclaimed
```

```
# Dirty:           123456 kB   ← modified but not yet written to disk
# Writeback:         1234 kB   ← currently being written to disk
# AnonPages:      3456789 kB   ← process heap/stack (not file-backed)
# Mapped:          234567 kB   ← mmap'd files (includes shared libs)
# SReclaimable:   1234567 kB   ← kernel slab cache (reclaimable)

# The dangerous ones to watch:
# Dirty → if very high, system is buffering lots of writes
# Writeback → if high, disk can't keep up with write-back rate
# SwapUsed → if non-zero, system was under memory pressure
```

🔷 Controlling dirty page writeback

```
# /proc/sys/vm/ — virtual memory tunables

# Writeback timing:
cat /proc/sys/vm/dirty_background_ratio    # Default: 10
# Background flusher starts when dirty pages > 10% of total RAM

cat /proc/sys/vm/dirty_ratio              # Default: 20
# Hard limit: writes BLOCK if dirty pages > 20% of total RAM

cat /proc/sys/vm/dirty_expire_centisecs   # Default: 3000 (30 seconds)
# Pages older than 30s are written back

cat /proc/sys/vm/dirty_writeback_centisecs # Default: 500 (5 seconds)
# Flusher wakes every 5 seconds

# For databases (reduce risk of data loss):
sudo sysctl vm.dirty_background_ratio=5
sudo sysctl vm.dirty_ratio=10
sudo sysctl vm.dirty_expire_centisecs=1000

# For write-heavy streaming (maximize throughput):
sudo sysctl vm.dirty_background_ratio=20
sudo sysctl vm.dirty_ratio=40

# Make persistent:
echo "vm.dirty_background_ratio=5" >> /etc/sysctl.conf
echo "vm.dirty_ratio=10" >> /etc/sysctl.conf
sudo sysctl -p
```

🔷 Dropping the page cache

```
# Drop page cache (free up RAM — Linux will repopulate as needed)
# Useful: before benchmarking, or to free RAM for a memory-hungry process
```

```
# Sync dirty pages first (important!)
sync

# Drop page cache only (1)
echo 1 > /proc/sys/vm/drop_caches

# Drop dentries and inodes (2)
echo 2 > /proc/sys/vm/drop_caches

# Drop page cache + dentries + inodes (3)
echo 3 > /proc/sys/vm/drop_caches
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'

# ⚠ Only do this for benchmarking or emergencies
# In production, Linux manages cache eviction automatically
# Dropping cache causes temporary performance degradation as cache refills

# ── vmstat — page cache behavior ──────────────────────────────────────

vmstat 1
# procs ---------memory---------- ---swap-- -----io---- -system-- ------
cpu-----
#  r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs  us sy
id  wa
#  2  0      0 1.1G   98M   9.8G    0    0  1234   567  123 456   2  1  95
2
#                                                ────  ───          ──
#                           bi=blocks read from disk  bo=blocks written
wa=waiting

# si/so = swap in/out — if non-zero: memory pressure, check for OOM risk
# wa = I/O wait % — high means CPU waiting for disk
# buff/cache growing = Linux using free RAM for cache (good!)
```

🔷 Short crisp interview answer

> "Linux aggressively uses free RAM as page cache for file data and buffer cache for filesystem metadata. The `free` command's 'available' column is what matters — not 'free' — because cached RAM is reclaimable on demand. Low 'free' with high 'available' means Linux is using RAM efficiently, not that you have a memory problem. I watch `/proc/meminfo`'s Dirty and Writeback values — high Dirty with rising Writeback means disks can't keep up with write-back. For databases, I tune `vm.dirty_ratio` lower to reduce write-back latency and risk of data loss on crash."

⚠ Gotchas

```
# GOTCHA 1: Low free memory is NORMAL on Linux
free -h
# free = 200MB    ← This looks alarming but...
```

```
# available = 12GB ← ...this is what apps can actually use
# Linux fills RAM with cache — this is the intended behavior

# GOTCHA 2: Dropping caches causes performance spike
echo 3 > /proc/sys/vm/drop_caches
# Next minute: cache miss storm — every read goes to disk
# Only do this before benchmarks, not on production load

# GOTCHA 3: high dirty_ratio + power loss = data loss
# With dirty_ratio=40: up to 40% of RAM may be dirty (not on disk)
# Power loss = potential loss of 40% of RAM worth of writes
# Databases use O_SYNC / fsync() to bypass this risk

# GOTCHA 4: mmap vs read — both use page cache
# mmap() maps file directly into process address space via page cache
# read() copies page cache data into userspace buffer
# Both benefit from caching — mmap avoids the copy (zero-copy)

# GOTCHA 5: Swap ≠ low memory
# Swap can be used even when RAM is available (kernel swaps out
# cold anonymous pages to keep RAM for file cache)
# vm.swappiness=10 reduces this tendency
sudo sysctl vm.swappiness=10   # Prefer keeping page cache over swapping
```

6.10 `fio` — I/O Benchmarking in Production

🔷 What `fio` is

`fio` (**F**lexible **I**/**O** Tester) is the standard tool for benchmarking disk performance. It can simulate any I/O pattern: sequential reads, random writes, mixed workloads, database-style access, etc. Used to characterize storage before deploying applications.

🔷 Why benchmarking matters

```
Without benchmarking, you're guessing whether storage can handle:
  - 10,000 IOPS for a database
  - 2GB/s sequential for video streaming
  - 500MB/s random writes for a cache tier

fio gives you measured ground truth:
  IOPS (I/O operations per second) → how many random 4KB reads per second
  Throughput (MB/s)                → how many MB of sequential data per
second
  Latency (ms/µs)                  → how long each I/O operation takes
  P99/P999 latency                 → tail latency (worst-case outliers)
```

🔷 Core `fio` usage

```
# Install: apt install fio

# fio can be used with command line flags or job files

# ── Sequential read throughput ──────────────────────────────────────

fio --name=seq-read \
    --rw=read \           # Sequential read
    --bs=1M \             # 1MB block size (sequential workload)
    --size=4G \           # Test file size (use ≥ RAM to bypass cache)
    --numjobs=1 \         # Single thread
    --iodepth=16 \        # 16 outstanding I/Os (queue depth)
    --direct=1 \          # Bypass page cache (tests actual disk speed)
    --filename=/data/fio_test

# Output:
# READ: bw=2456MiB/s (2575MB/s), iops=2456, run=1677-1677msec
# lat (usec): min=385, max=8192, avg=516.3, stdev=234.1

# ── Sequential write throughput ─────────────────────────────────────

fio --name=seq-write \
    --rw=write \
    --bs=1M \
    --size=4G \
    --numjobs=1 \
    --iodepth=16 \
    --direct=1 \
    --filename=/data/fio_test

# ── Random read IOPS (database read simulation) ────────────────────

fio --name=rand-read \
    --rw=randread \       # Random read pattern
    --bs=4k \             # 4KB blocks (typical database page size)
    --size=4G \
    --numjobs=4 \         # 4 parallel jobs (simulate concurrent queries)
    --iodepth=32 \        # Deep queue (simulate many concurrent I/Os)
    --direct=1 \
    --filename=/data/fio_test

# Output:
# READ: bw=1457MiB/s (1528MB/s), iops=373400, run=10001-10001msec
# lat (usec): min=32, max=2847, avg=85.6, stdev=124.3
#   clat percentiles:
#    | 50.00th=[   84],  75.00th=[   97]
#    | 90.00th=[  114], 95.00th=[  131]
#    | 99.00th=[  245], 99.50th=[  363]  ← P99 = worst 1% of requests
#    | 99.90th=[  906], 99.99th=[ 2737]  ← P999 = worst 0.1%
```

```
# ── Random write IOPS ──────────────────────────────────────────

fio --name=rand-write \
    --rw=randwrite \
    --bs=4k \
    --size=4G \
    --numjobs=4 \
    --iodepth=32 \
    --direct=1 \
    --filename=/data/fio_test

# ── Mixed read/write (75% read, 25% write — OLTP simulation) ──────────

fio --name=mixed \
    --rw=randrw \         # Mixed random read + write
    --rwmixread=75 \      # 75% reads, 25% writes
    --bs=4k \
    --size=4G \
    --numjobs=4 \
    --iodepth=32 \
    --direct=1 \
    --filename=/data/fio_test

# ── Latency test (sync writes — database commit simulation) ──────────

fio --name=sync-write \
    --rw=randwrite \
    --bs=4k \
    --size=1G \
    --numjobs=1 \
    --iodepth=1 \         # Queue depth 1 = synchronous (one at a time)
    --direct=1 \
    --fsync=1 \           # fsync after every write (database commit
behavior)
    --filename=/data/fio_test

# This measures: how many database transactions per second
# Good NVMe: ~100,000 IOPS
# Good SSD:  ~10,000-50,000 IOPS
# Good HDD:  ~100-200 IOPS (horrible for databases!)
```

🔷 fio job files

```
# More maintainable than command line for complex tests
cat /tmp/db_benchmark.fio

[global]
ioengine=libaio          # Linux async I/O engine (most realistic)
direct=1                 # Bypass page cache
size=8G                  # Test file size
```

```
runtime=60                # Run for 60 seconds
time_based                # Run for runtime, not until size exhausted
group_reporting           # Combine stats across jobs

[random-reads]
rw=randread
bs=4k
numjobs=4
iodepth=32
filename=/data/fio_test1

[random-writes]
rw=randwrite
bs=4k
numjobs=2
iodepth=16
filename=/data/fio_test2

# Run the job file:
sudo fio /tmp/db_benchmark.fio

# ── I/O engines ──────────────────────────────────────────────────

# ioengine=sync      → synchronous read/write (simplest)
# ioengine=libaio    → Linux async I/O (most realistic for databases)
# ioengine=io_uring  → Modern async I/O (best for NVMe, Linux 5.1+)
# ioengine=psync     → pread/pwrite (thread-safe, no async)
# ioengine=mmap      → memory-mapped I/O

fio --ioengine=io_uring ...   # Best for NVMe benchmarking
```

🔷 Interpreting fio output

```
# Full output example:
fio --name=randread --rw=randread --bs=4k --size=4G \
    --numjobs=4 --iodepth=32 --direct=1 --filename=/data/test

# randread: (g=0): rw=randread, bs=(R) 4096B-4096B, (W) 4096B-4096B,
ioengine=libaio, iodepth=32
# ...
# Jobs: 4 (f=4): [r(4)][100.0%][r=1584MiB/s][r=405k IOPS][eta 00m:00s]
#
# randread: (groupid=0, jobs=4): err= 0: pid=12345: ...
#   read: IOPS=404952, BW=1582MiB/s (1659MB/s)(94.9GiB/60001msec)
#   ────  ────────────  ────────────
#        I/O ops/sec   Throughput
#
#     clat (usec): min=72, max=12345, avg=315.4, stdev=189.2
#     ──────────────────────────────────────────────
#     clat = completion latency (time from I/O submission to completion)
```

```
#
#      lat (usec): min=72, max=12346, avg=315.7, stdev=189.2
#      ──────────────────────────────────────────────────
#      lat = total latency (includes queue wait)
#
#     clat percentiles (usec):
#      |  1.00th=[  104],  5.00th=[  147], 10.00th=[  178]
#      | 20.00th=[  215], 30.00th=[  249], 40.00th=[  277]
#      | 50.00th=[  302], 60.00th=[  330], 70.00th=[  363]
#      | 80.00th=[  404], 90.00th=[  461], 95.00th=[  515]
#      | 99.00th=[  660], 99.50th=[  734], 99.90th=[  938]
#      | 99.99th=[ 2114]

# Key numbers:
# IOPS: 404,952 → NVMe class SSD (excellent)
# P50 latency: 302µs
# P99 latency: 660µs  ← 99% of requests complete in under 660µs
# P999 latency: 938µs ← 99.9% complete in under 938µs
# P9999 latency: 2114µs ← even worst-case is reasonable
```

🔷 Reference benchmarks

```
# Expected performance (rough guides):
#
#  ┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐
#  │ Storage Type     │ Seq Read         │ Random 4K Read   │ Sync Write       │
#  │                  │ (MB/s)           │ (IOPS)           │ IOPS             │
#  ├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
#  │ 7200rpm HDD      │ 100-200          │ 75-150           │ 100-200          │
#  │ SATA SSD         │ 500-600          │ 50k-100k         │ 10k-50k          │
#  │ NVMe SSD         │ 3000-7000        │ 500k-1M+         │ 100k-500k        │
#  │ AWS EBS gp3      │ 125-1000         │ 3k-16k           │ 3k-16k           │
#  │ AWS EBS io2      │ up to 4000       │ up to 64k        │ up to 64k        │
#  │ AWS Instance     │ varies           │ up to 2M         │ high             │
#  │ Store (NVMe)     │                  │                  │                  │
#  └──────────────────┴──────────────────┴──────────────────┴──────────────────┘

# Cloud storage IOPS limits (AWS gp3 default):
# gp3: 3000 IOPS, 125 MB/s baseline
# gp3: up to 16000 IOPS, 1000 MB/s (provisioned, costs more)
# io2: up to 64000 IOPS per volume

# ── Production pre-deployment benchmark ──────────────────────────────

#!/bin/bash
# Run before deploying database to validate storage
TESTFILE="/data/fio_benchmark"
RUNTIME=60

echo "=== Sequential Read ==="
```

```
fio --name=seq-read --rw=read --bs=1M --size=8G \
    --numjobs=1 --iodepth=8 --direct=1 --runtime=$RUNTIME \
    --time_based --filename=$TESTFILE --output-format=terse \
    | awk -F';' '{print "Read MB/s:", $7}'

echo "=== Random Read IOPS ==="
fio --name=rand-read --rw=randread --bs=4k --size=8G \
    --numjobs=4 --iodepth=32 --direct=1 --runtime=$RUNTIME \
    --time_based --filename=$TESTFILE --output-format=terse \
    | awk -F';' '{print "Read IOPS:", $8}'

echo "=== Sync Write (DB commit latency) ==="
fio --name=sync-write --rw=randwrite --bs=4k --size=1G \
    --numjobs=1 --iodepth=1 --direct=1 --fsync=1 \
    --runtime=$RUNTIME --time_based --filename=$TESTFILE \
    --output-format=terse \
    | awk -F';' '{print "Write IOPS:", $49, "P99 lat (us):", $90}'

rm -f $TESTFILE
```

🔷 Short crisp interview answer

> "`fio` is the standard storage benchmarking tool — I use it to characterize storage before deploying databases. The three key benchmarks: sequential throughput with 1MB blocks and `--rw=read/write`; random IOPS with 4KB blocks `--rw=randread`; and sync write latency with `--iodepth=1 --fsync=1` which simulates database transaction commits. Always use `--direct=1` to bypass the page cache — otherwise you're benchmarking RAM. For databases, P99 and P999 latency matter more than average latency — a slow outlier causes query timeouts. NVMe should give <1ms P99 for 4KB random reads."

⚠ Gotchas

```
# GOTCHA 1: Not using --direct=1
fio --rw=randread --bs=4k ...    # Without --direct=1
# After the file fits in page cache → reading from RAM, not disk
# Results: 10M IOPS — that's your RAM, not your disk!
# Always: --direct=1 for storage benchmarking

# GOTCHA 2: Test file smaller than RAM
fio --size=500M --rw=read ...    # On server with 16GB RAM
# After first pass: entire test file in page cache → meaningless
# Rule: test file should be ≥ 2× RAM, or use --direct=1

# GOTCHA 3: Benchmarking in a VM without knowing the host
# Cloud VMs have I/O credit bursting — short benchmarks show burst speed
# Use: --runtime=300 (5 minutes) to exhaust burst credits
# Results after burst expiry = sustained performance you'll actually get
```

```
# GOTCHA 4: Not matching workload pattern
# Sequential benchmark on a database workload = wrong answer
# Match: database → 4K random; video streaming → 1M sequential
# Mixed OLTP → randrw with rwmixread=70

# GOTCHA 5: numjobs × iodepth = total outstanding I/Os
# numjobs=4, iodepth=32 → 128 outstanding I/Os at once
# NVMe can handle high queue depth; HDD saturates at low queue depth
# For HDDs: iodepth=1-4 is more realistic
```

🏆 Category 6 — Complete Mental Model

```
STORAGE TROUBLESHOOTING FLOW
═══════════════════════════════════

Problem: "Disk is full"
    │
    ├─ 1. Which filesystem?
    │       df -h | awk '$5+0 >= 90'
    │
    ├─ 2. Data blocks OR inodes?
    │       df -i | awk '$5+0 >= 90'
    │
    ├─ 3. What's using the space?
    │       du -sh /var/*/ | sort -rh | head -20
    │
    ├─ 4. Any deleted-but-open files?
    │       lsof +L1
    │       → restart process to free space
    │
    └─ 5. Which files are largest?
            find /var -type f -size +100M -ls

Problem: "I/O is slow"
    │
    ├─ 1. Which disk is saturated?
    │       iostat -hx 2 → watch %util and await
    │
    ├─ 2. Which process is causing it?
    │       sudo iotop -o
    │
    ├─ 3. Which files?
    │       lsof -p <pid>
    │
    ├─ 4. Is the scheduler optimal?
    │       cat /sys/block/sda/queue/scheduler
    │       cat /sys/block/sda/queue/rotational
    │
```

```
    └─ 5. Baseline: what should performance be?
            fio --name=test --rw=randread --bs=4k ...

TOOL SELECTION:
═══════════════════════

Disk space used?           → df -h
Directory space used?      → du -sh
Inode exhaustion?          → df -i
Block devices & UUIDs?     → lsblk -f, blkid
Partition management?      → parted (script), fdisk (interactive)
LVM management?            → pvs/vgs/lvs, lvextend -r
I/O performance?           → iostat -hx 2
Who's doing I/O?           → sudo iotop -o
Filesystem type trade-off? → ext4 (safe), xfs (scale), btrfs (snapshots)
Storage benchmarking?      → fio --direct=1
Page cache pressure?       → free -h, /proc/meminfo (Dirty, Available)
RAID health?               → cat /proc/mdstat, mdadm --detail /dev/md0
```

⚠ Master Gotcha List

#	Gotcha	Reality
1	Disk shows space available but can't create files	Inode exhaustion — check `df -i`
2	Deleted file but disk still full	Process holds fd open — `lsof +L1`, restart process
3	`free` shows low free memory	Check `available` column — cache is reclaimable
4	Bad `/etc/fstab` entry	Server won't boot — test with `sudo mount -a` first
5	Device name `/dev/sda` changed at boot	Always use UUID in fstab — `blkid` to find
6	LVM snapshot invalidated	COW buffer full — monitor `lvs -o lv_name,data_percent`
7	xfs cannot shrink	Plan LV size upfront; use ext4 if shrink needed
8	RAID is not a backup	RAID 1 mirrors deletions too — maintain separate backups
9	RAID 5 rebuild second failure	Size risk with large HDDs — prefer RAID 6 or RAID 10
10	fio benchmarks RAM not disk	Always use `--direct=1` to bypass page cache
11	Cloud storage burst IOPS	Run fio for ≥ 5 min to see sustained throughput
12	tmpfs uses RAM	Set `size=` limits or risk OOM

🔥 Top Interview Questions

Q1: Disk is 100% full but `du -sh /` shows less than `df -h`. Why?

> Deleted files held open by processes still consume blocks. The filesystem marks the blocks used until the last file descriptor is closed. `lsof +L1` lists these zombie file descriptors. The fix is to restart the process holding the fd open, or truncate the file with `> /file` while the process keeps it open.

Q2: You have space on disk but can't create new files. What's wrong?

> Inode exhaustion. Every file needs one inode. `df -i` will show 100% inode usage. Common causes: millions of small files in PHP session dirs, Node.js node_modules, or mail queue overflow. Fix: find the directory with too many files using `du --inodes -s /var/*/`, then clean up.

Q3: Explain the difference between ext4, xfs, and btrfs for a database workload.

> For databases, I'd choose xfs or ext4. xfs handles parallel I/O better through allocation groups — it's the RHEL default specifically for high-throughput workloads. ext4 is solid and universal. I'd avoid btrfs for databases because Copy-on-Write is inefficient for small random writes — every write copies the old block, writes to a new location, and updates metadata. You'd also want to disable CoW with `chattr +C` on database data directories if you do use btrfs.

Q4: How does LVM allow you to resize a volume without downtime?

> You add a Physical Volume to the Volume Group with `vgextend`, then extend the Logical Volume with `lvextend -L +50G /dev/vg0/lv_data`. Finally, grow the filesystem: `resize2fs` for ext4 or `xfs_growfs` for xfs — both work while mounted. The `-r` flag to `lvextend` does both steps automatically. The key insight is that LVM's logical block addresses are remapped to physical extents in metadata — adding space just updates the mapping table.

Q5: What does `%util=100%` in iostat mean, and is it always a problem?

> `%util` measures what fraction of time the device had at least one outstanding I/O request. For HDDs, 100% truly means saturation. For SSDs and NVMe, it's misleading — they can process many I/Os in parallel (high queue depth), so `%util=100%` might mean 10% actual utilization. For SSDs, watch `aqu-sz` (queue depth) and `await` (latency) instead — if latency is still low at 100% util, the device isn't actually bottlenecking.

Q6: What's the difference between the page cache and swap?

> Page cache stores recently accessed file data in RAM for fast re-reads — this is always beneficial. Swap stores anonymous memory (process heap/stack) on disk when RAM is tight — this is always bad for performance. The key: page cache pages are clean copies of disk data and can be dropped immediately when RAM is needed. Swap pages are the only copy of data and must be written back before the RAM can be reused.

This document covers all 10 topics in Category 6: Storage & I/O — from basic disk usage tools through advanced I/O schedulers, page cache internals, and fio benchmarking, with production debugging patterns throughout.